

Departamento de Eletrónica, Telecomunicações e
Informática

LECTURE 4: NEURAL NETWORKS

Petia Georgieva
(petia@ua.pt)

NEURAL NETWORKS- outline

- 1. NN - non-linear classifier**
- 2. Neuron model: logistic unit**
- 3. NN - binary versus multi-class classification**
- 4. Cost function (with or without regularization)**
- 5. NN learning - Error Backpropagation algorithm**

Classification of non-linearly separable data

x_1 = size of house
 x_2 = no. of bedrooms
 x_3 = no. of floors
 x_4 = age of house
 x_5 = average income in neighborhood
 x_6 = kitchen size
 $\vdots$
 x_{100}

Let we have 100 original features:

If using quadratic combinations of the features to get nonlinear decision boundary, we end up with 5000 features

Logistic regression is not efficient for such complex nonlinear models.

Computer vision: car detection



Cars



Not a car

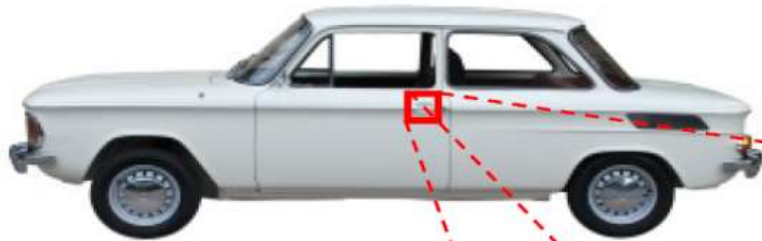
Testing:



What is this?

Computer vision

You see this:



But the camera sees this:

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

For a small piece of the car image we may have too many features (pixels)

Computer vision: object detection

50 x 50 pixel images $\rightarrow$ 2500 pixels
 $n = 2500$ (7500 if RGB)

$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

50 x 50 pixel images $\Rightarrow$

2500 pixels (features) for a gray scale image

7500 pixels (features) for a RGB image

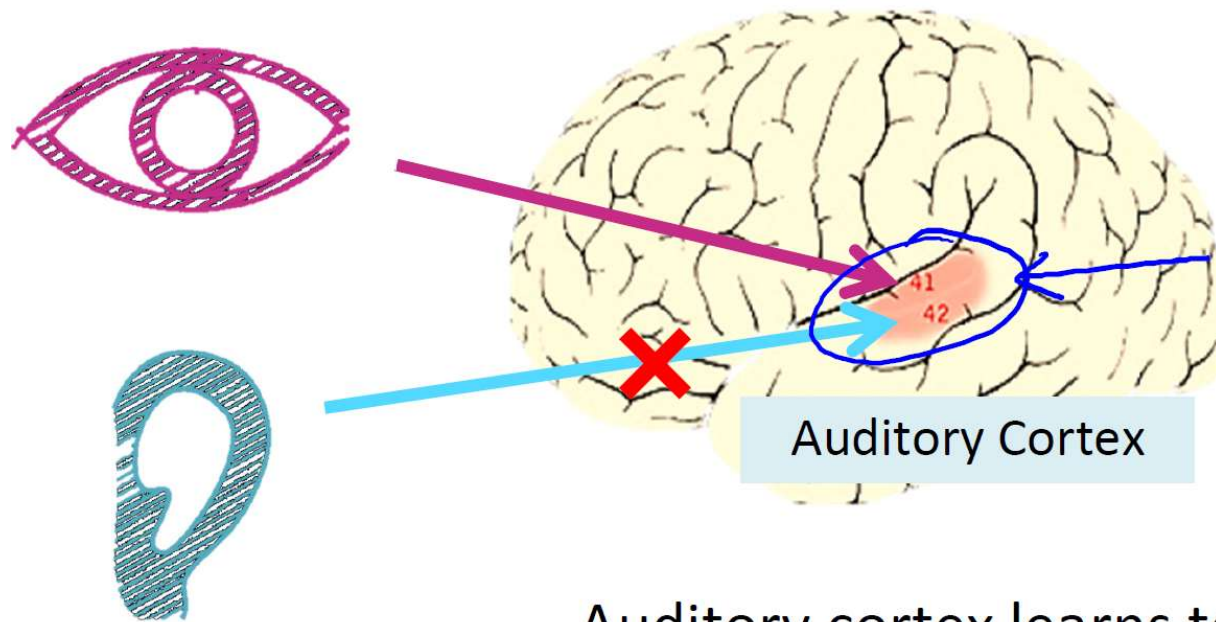
If using quadratic features $\Rightarrow$ 3 million features

Logistic regression is not suitable for such complex nonlinear models.

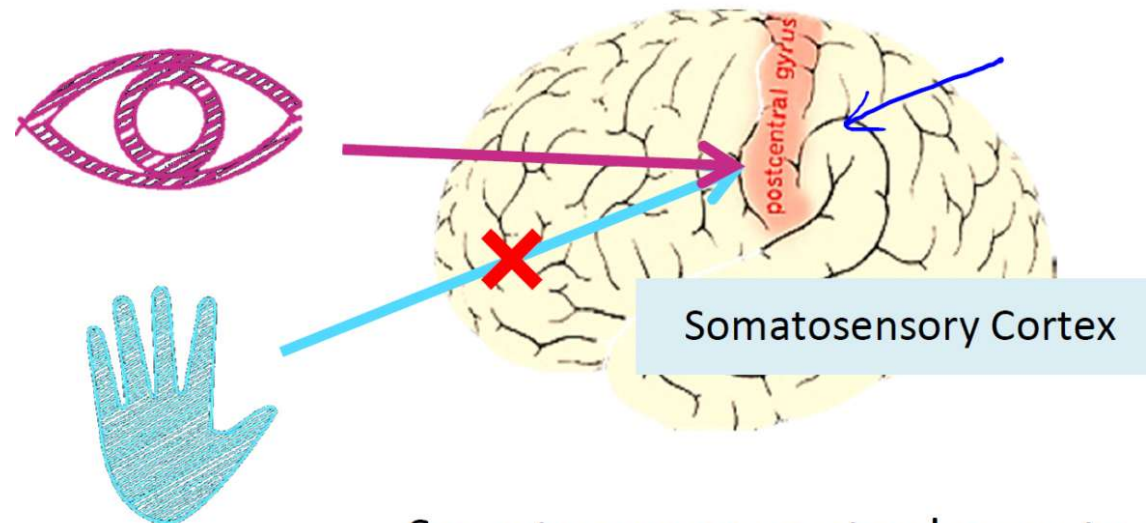
Neural Networks fit better complex nonlinear models.

Brain experiments

(brain can learn from any sensor wired to it)



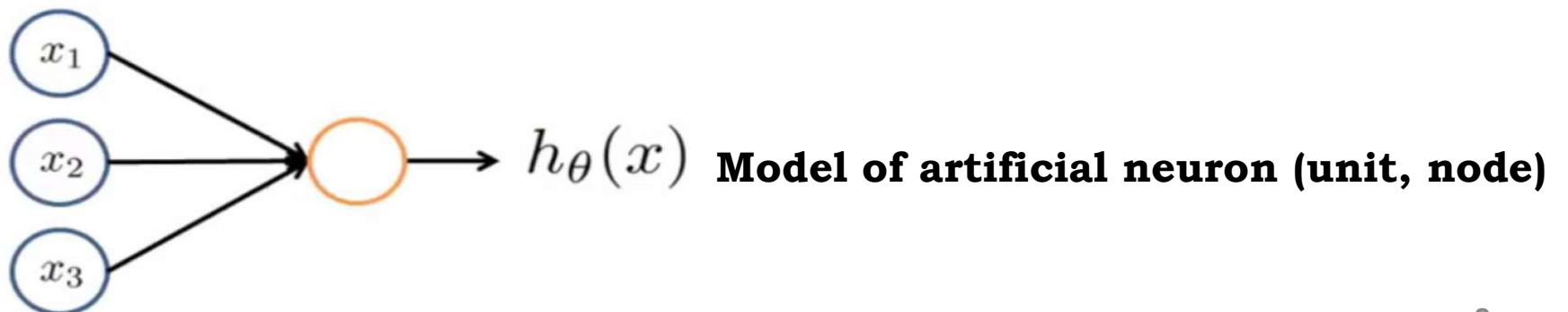
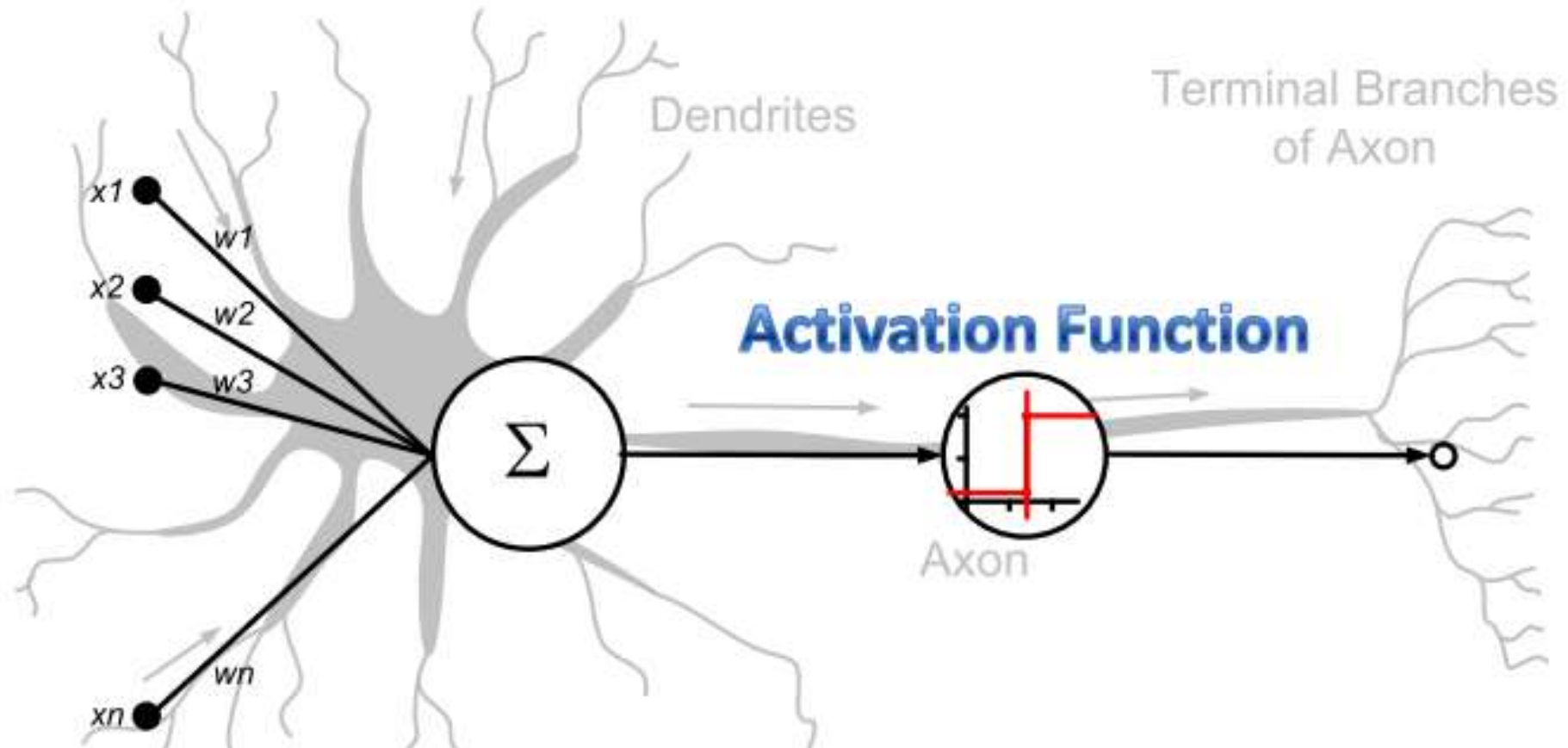
Auditory cortex learns to see



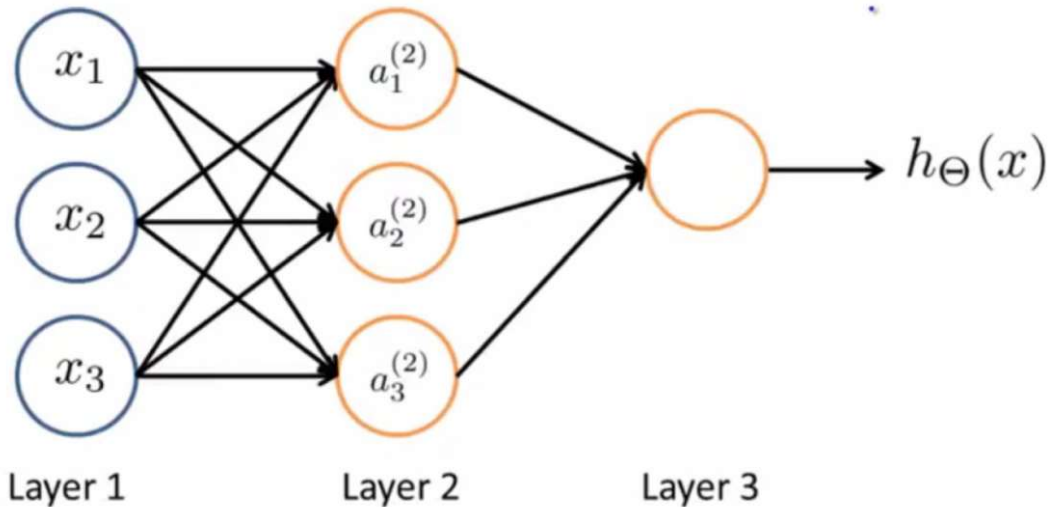
Somatosensory cortex learns to see

Neuron model

Origins: NN models inspired by biological neuron structures and computations.



Neural Network



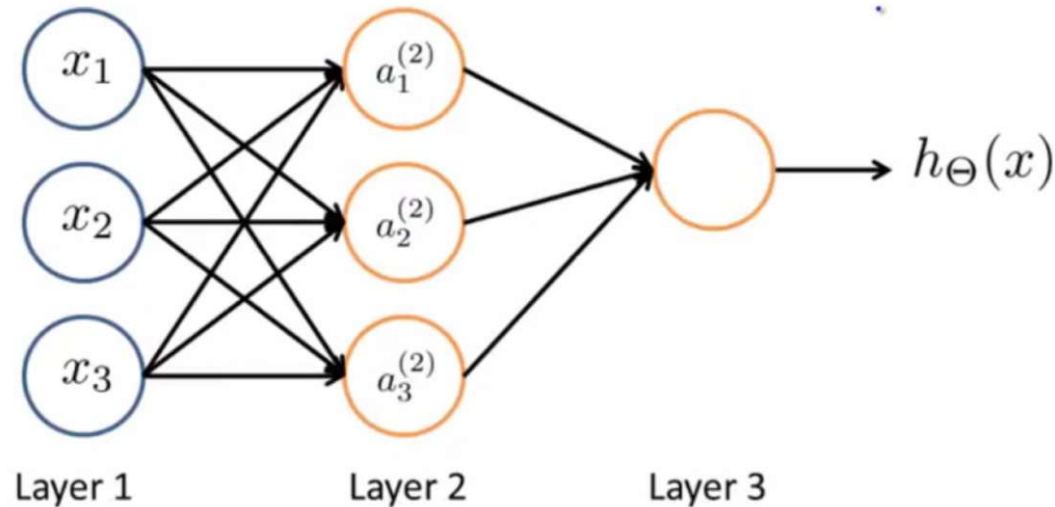
Input layer hidden layer output layer

$a_i^{(j)}$ = "activation" of unit i in layer j
 $\Theta^{(j)}$ = matrix of weights controlling
 function mapping from layer j to
 layer $j + 1$

$$\begin{aligned} a_1^{(2)} &= g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3) \\ a_2^{(2)} &= g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3) \\ a_3^{(2)} &= g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3) \\ h_{\Theta}(x) &= a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)}) \end{aligned}$$

If network has s_j units in layer j , s_{j+1} units in layer $j + 1$, then $\Theta^{(j)}$ will be of dimension $s_{j+1} \times (s_j + 1)$.

Neural Network –vectorized implementation



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$z^{(2)} = \Theta^{(1)} x$$

$$a^{(2)} = g(z^{(2)})$$

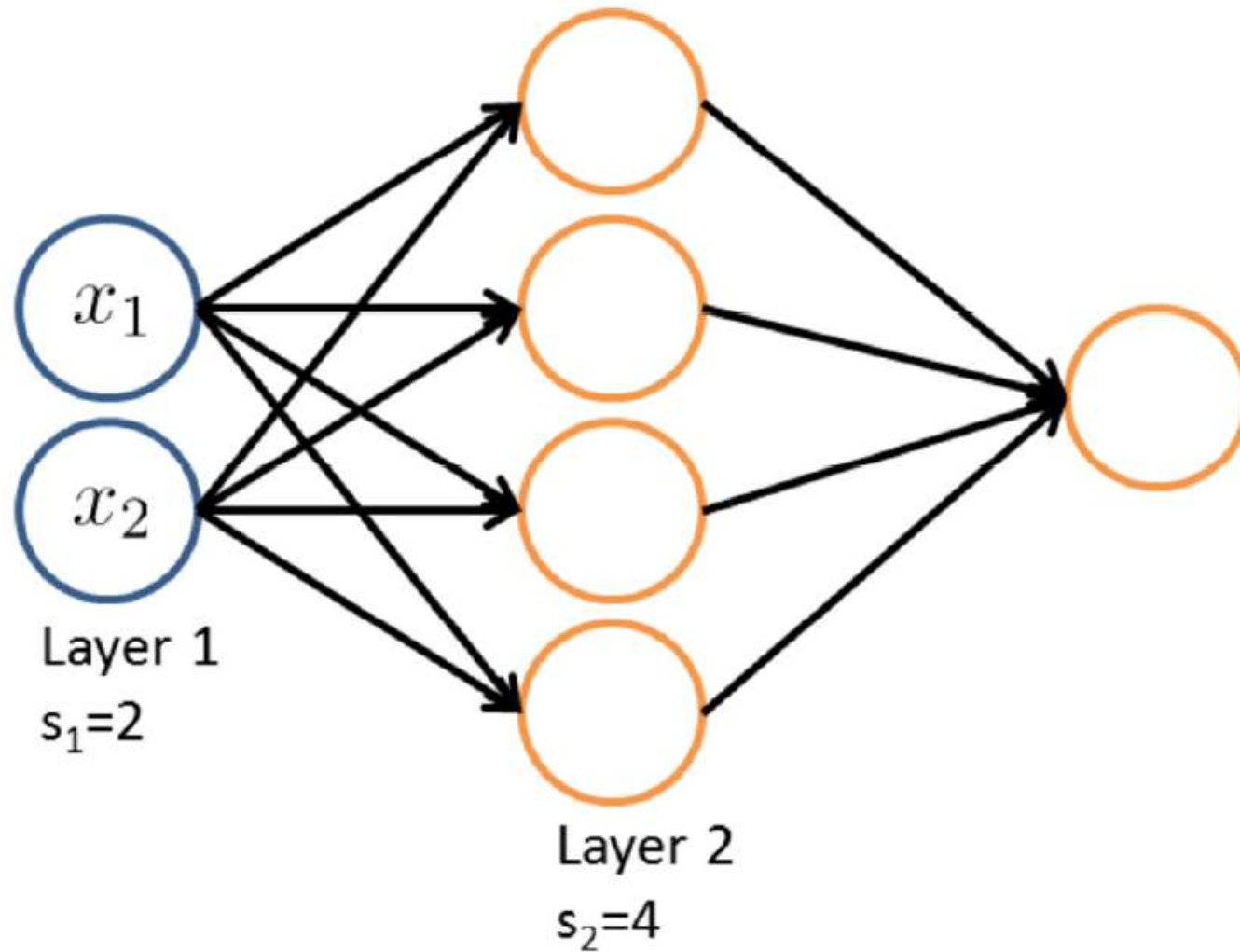
$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

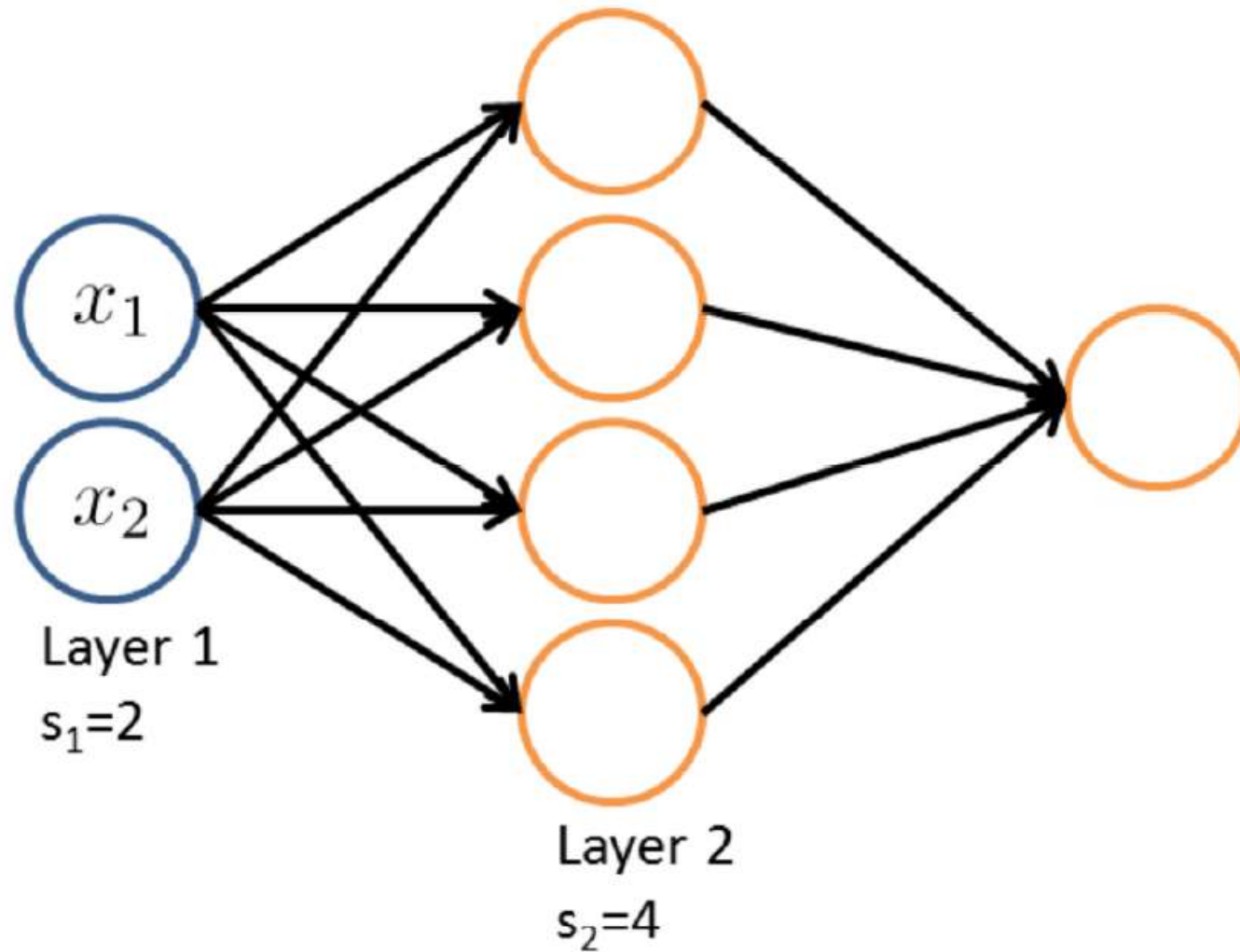
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

**Question: how many weight matrices has the NN
and what is the dymension of each matrix ?**



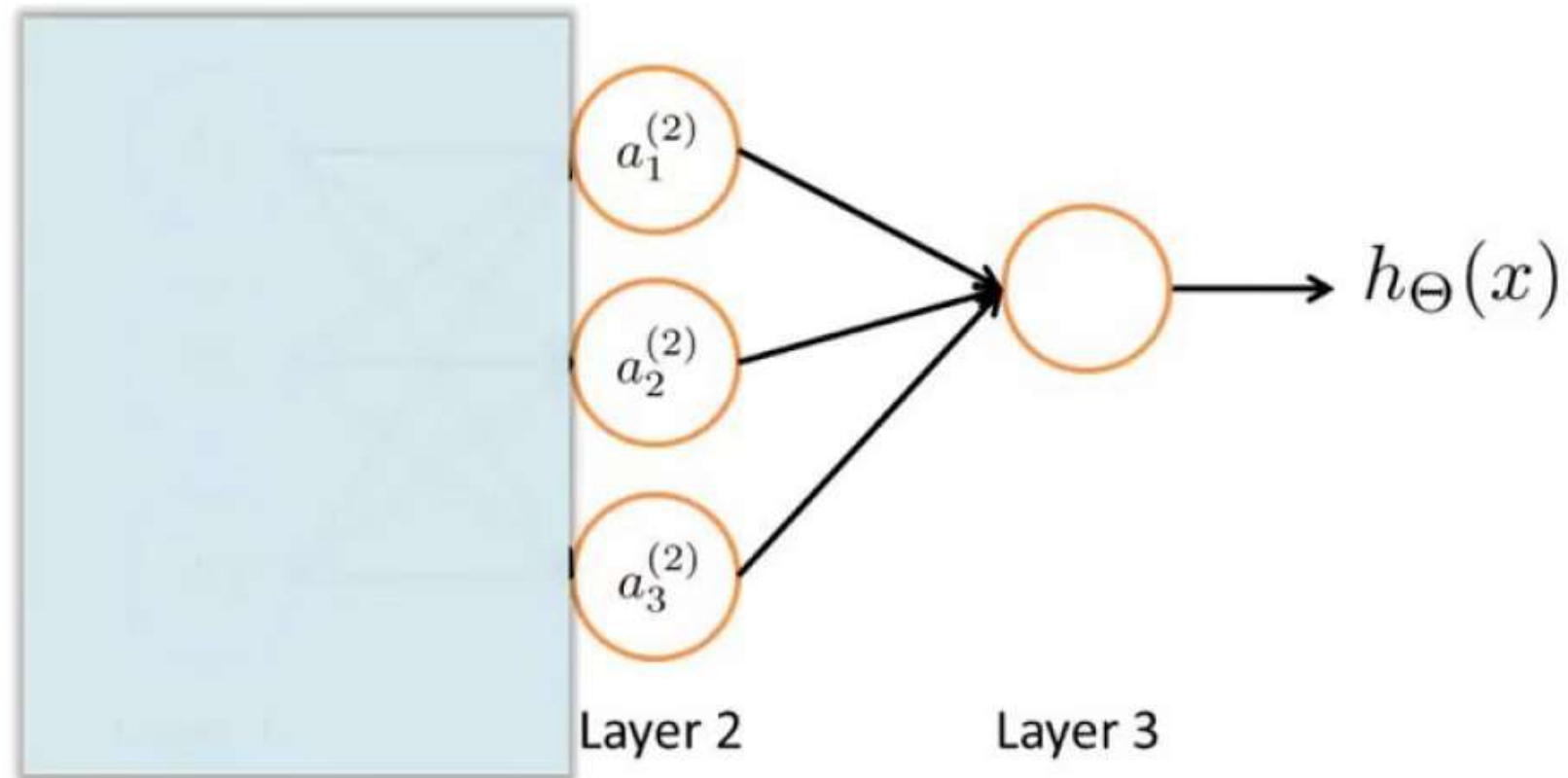
**Question: how many weight matrices has the NN
and what is the dymension of each matrix ?**



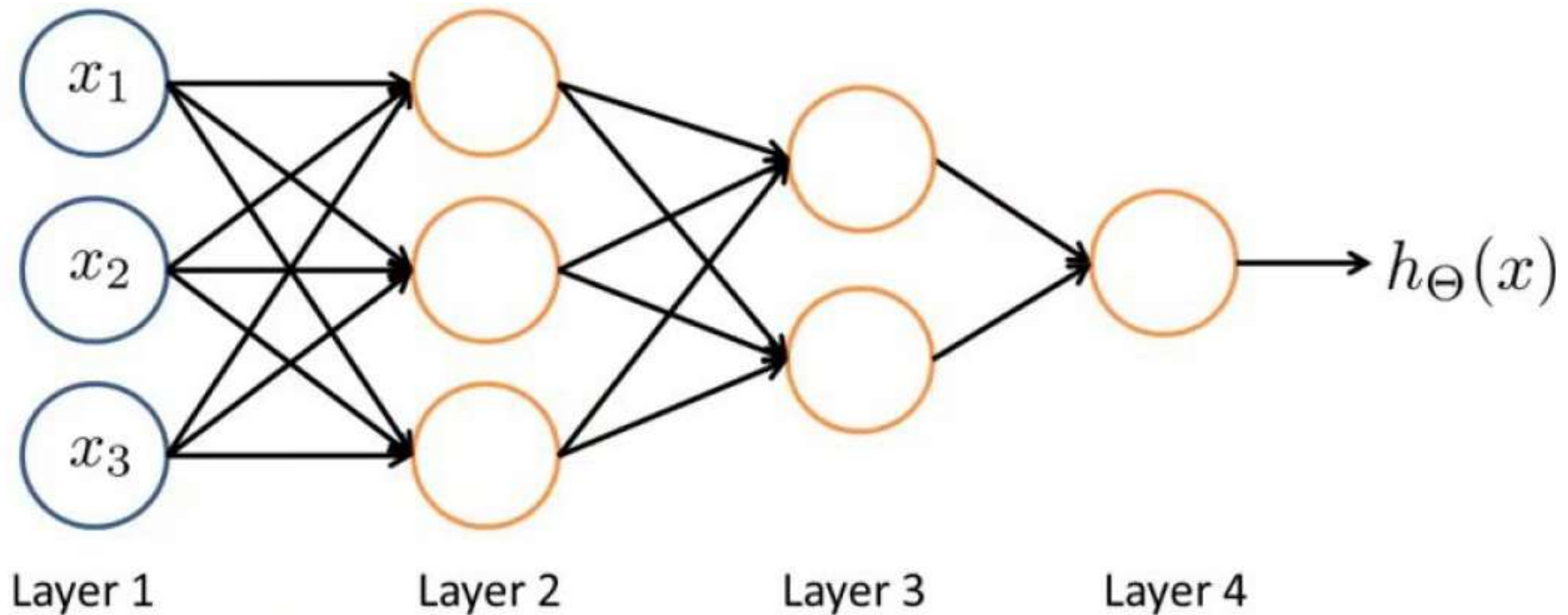
Q1=> 4×3

Q2=> 1×5

Neural Network is learning its own features

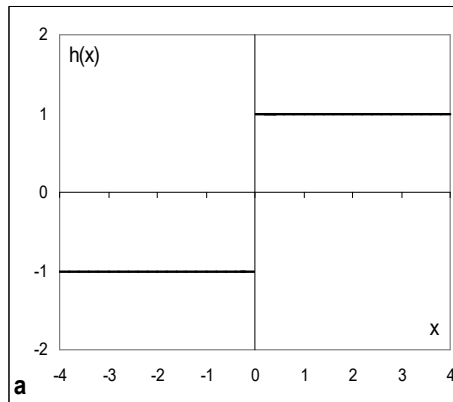
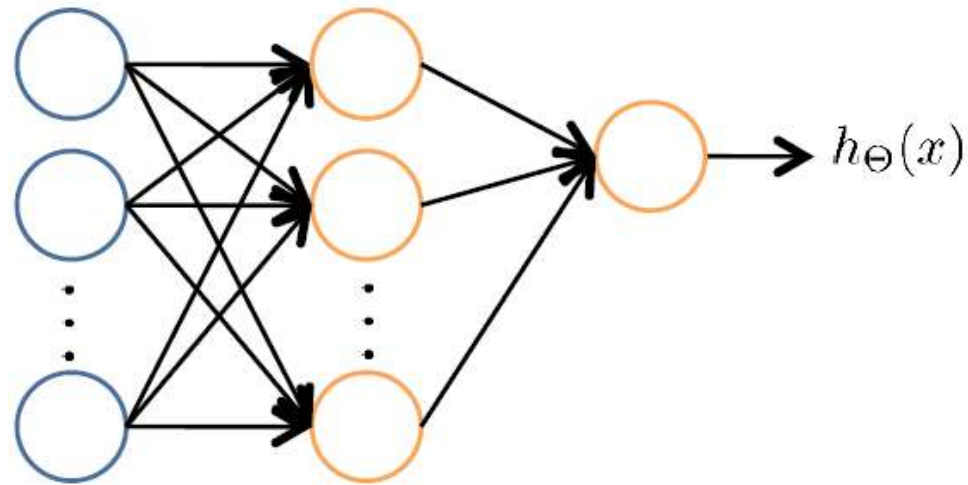


Other Network Architectures

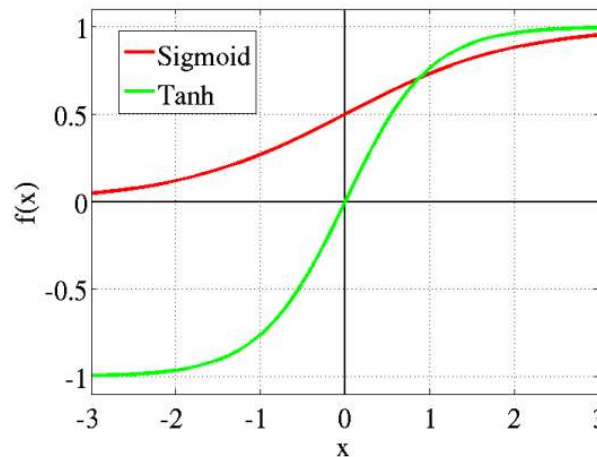


Many hidden layers can build more complex functions of the inputs (the data) $\Rightarrow$ NN can learn pretty complex functions $\Rightarrow$ **deep learning**

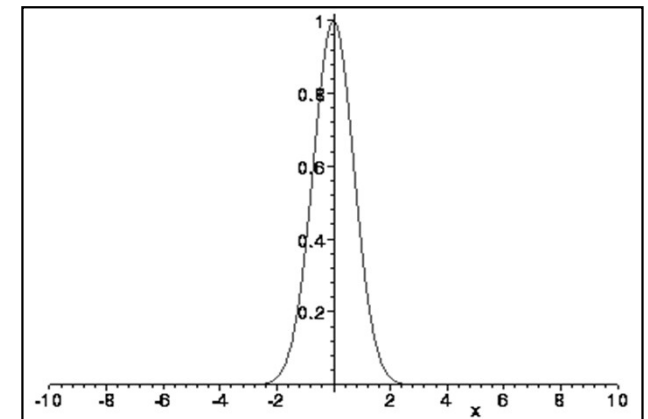
Typical Activation functions



Step (heaviside)

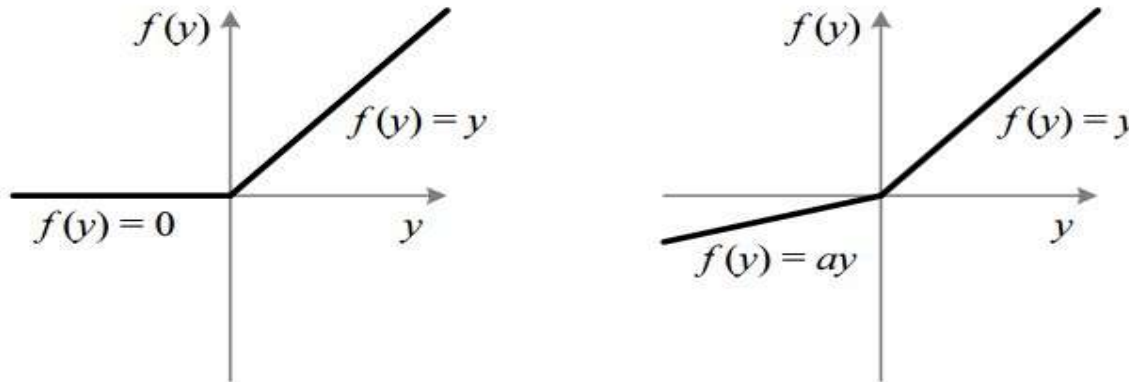


Sigmoid (logistic) vs. Hyperbolic tangent (Tanh)



Radial Basis Function (RBF)

Typical Activation functions



ReLU (Rectified Linear Unit) vs. Leaky ReLU

ReLU:

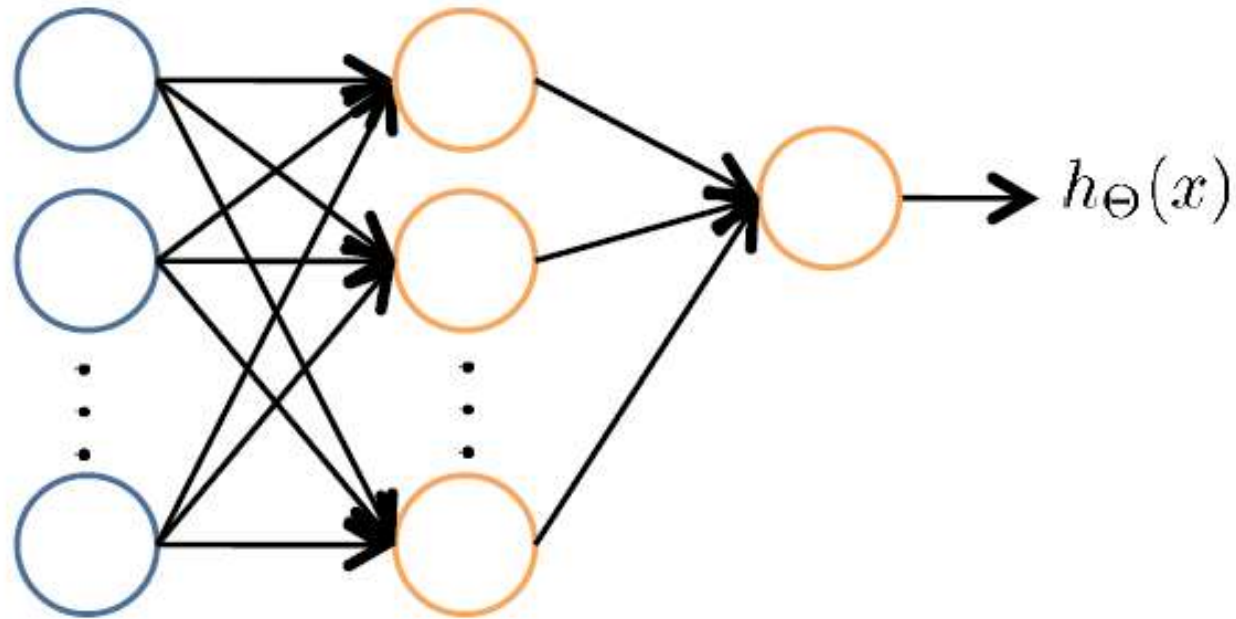
- + Computationally efficient—the network training can converge faster
- + Non-linear (though it looks like a linear function), it is easy to compute the ReLU derivative => suitable to be used for backpropagation.
- Dying ReLU problem—when inputs approach zero, or are negative, ReLU gradient = 0, the network cannot perform backpropagation and cannot learn.

Leaky ReLU:

- + Prevents dying ReLU problem—this variation of ReLU has a small positive slope in the negative area, so it does enable backpropagation, even for negative input values.

Softmax: handles multiple classes, has as many outputs as classes. The value of each output is the probability of the class. The sum of all softmax outputs = 1.

NN - binary classification



Training set: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

2 classes { 0,1 } => one output unit

NN - multi-class classification



Pedestrian



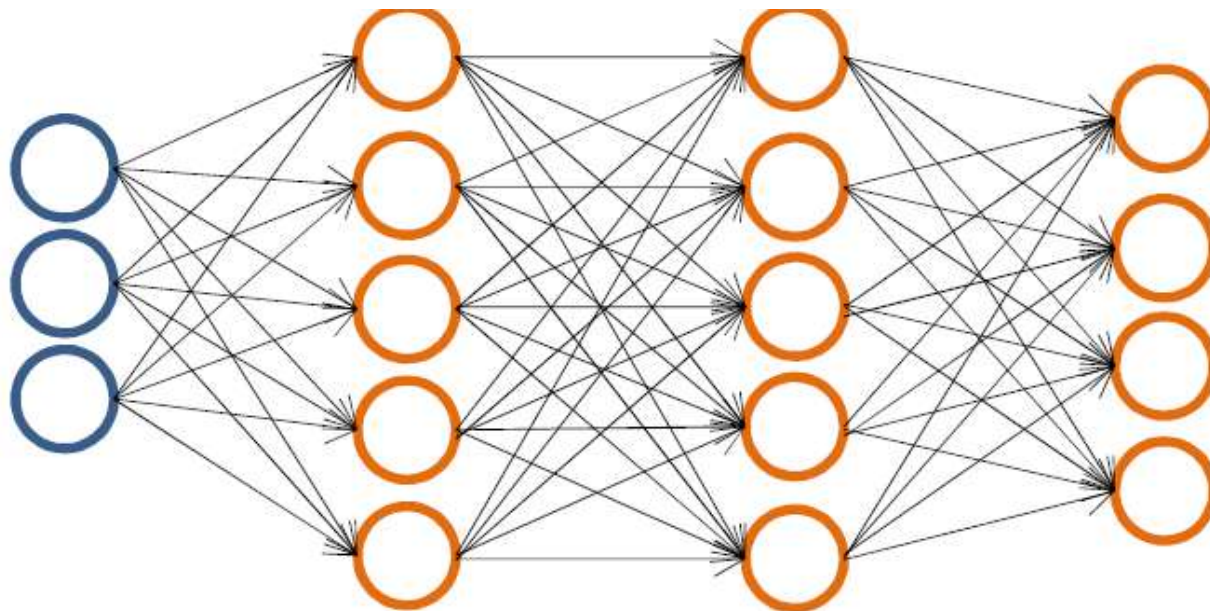
Car



Motorcycle



Truck



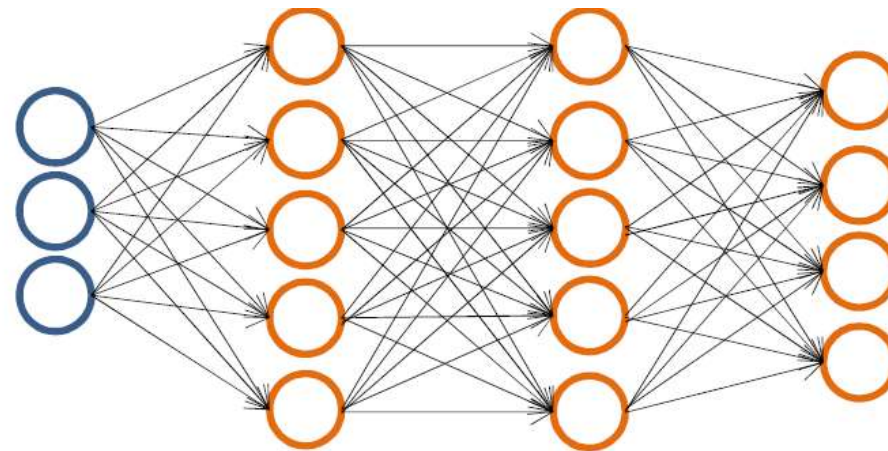
$$h_{\Theta}(x) \in \mathbb{R}^4$$

K classes {1,2, K} => K output units

Multiple output units: One versus all

Training set: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

$y^{(i)}$ one of $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$



$h_{\Theta}(x) \in \mathbb{R}^4$

Want $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.
when pedestrian when car when motorcycle

NN Cost Functions (without regularization)

Logistic Regression (Binary cross entropy loss function) :

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

NN with 1 output (logistic) unit (suitable for binary classification):
(the same as log regression)

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

NN Cost Functions (without regularization)

NN with 1 output (logistic) unit (suitable for binary classification problems):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

NN with K output (logistic) units (suitable for multiclass classif. problems):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left(\sum_{k=1}^K \left[-y_k^{(i)} \log((h_{\theta}(x^{(i)}))_k) - (1 - y_k^{(i)}) \log(1 - (h_{\theta}(x^{(i)}))_k) \right] \right)$$

NN with 1 output (not logistic) suitable for nonlinear regression problems:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Cost Function with regularization

Regularized Logistic Regression:

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Regularization term

Neural Network with K output (logistic) units:

$$h_{\Theta}(x) \in \mathbb{R}^K \quad (h_{\Theta}(x))_i = i^{th} \text{ output}$$

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right]$$

Regularization term

$$+ \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

L = total no. of layers in network

s_l = no. of units (not counting bias unit) in layer l

NN classification - example

MNIST handwritten digit dataset (<http://yann.lecun.com/exdb/mnist/>).
5000 training examples (28x28 pixels image, indicating the grayscale color intensity). The image is transformed into a row vector (with 784 elements). This gives 5000 x 784 data matrix X (every row is a training example).



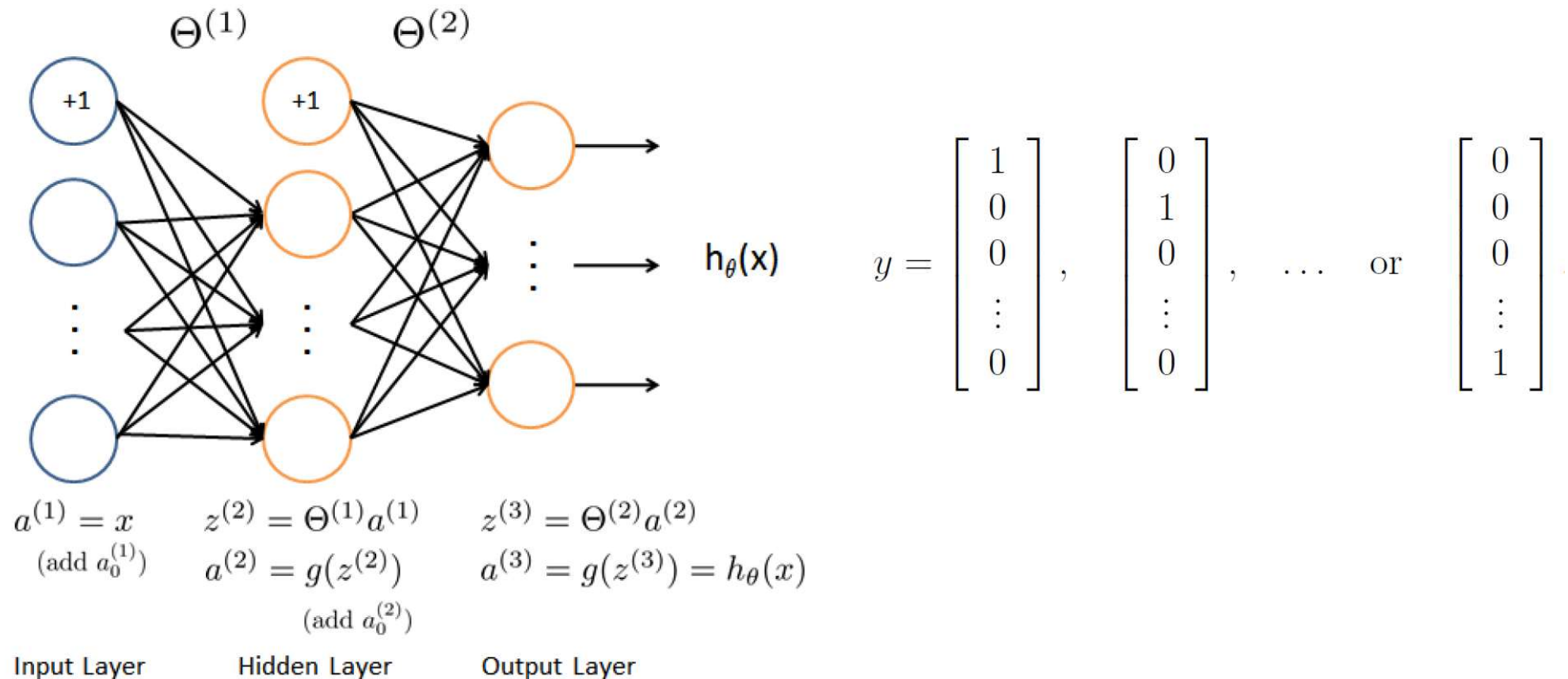
NN model - example

input layer – 400 units = 20x20 pixels (input features) + 1 unit(=1, the bias)

hidden layer – 25 units + 1 unit(=1, the bias)

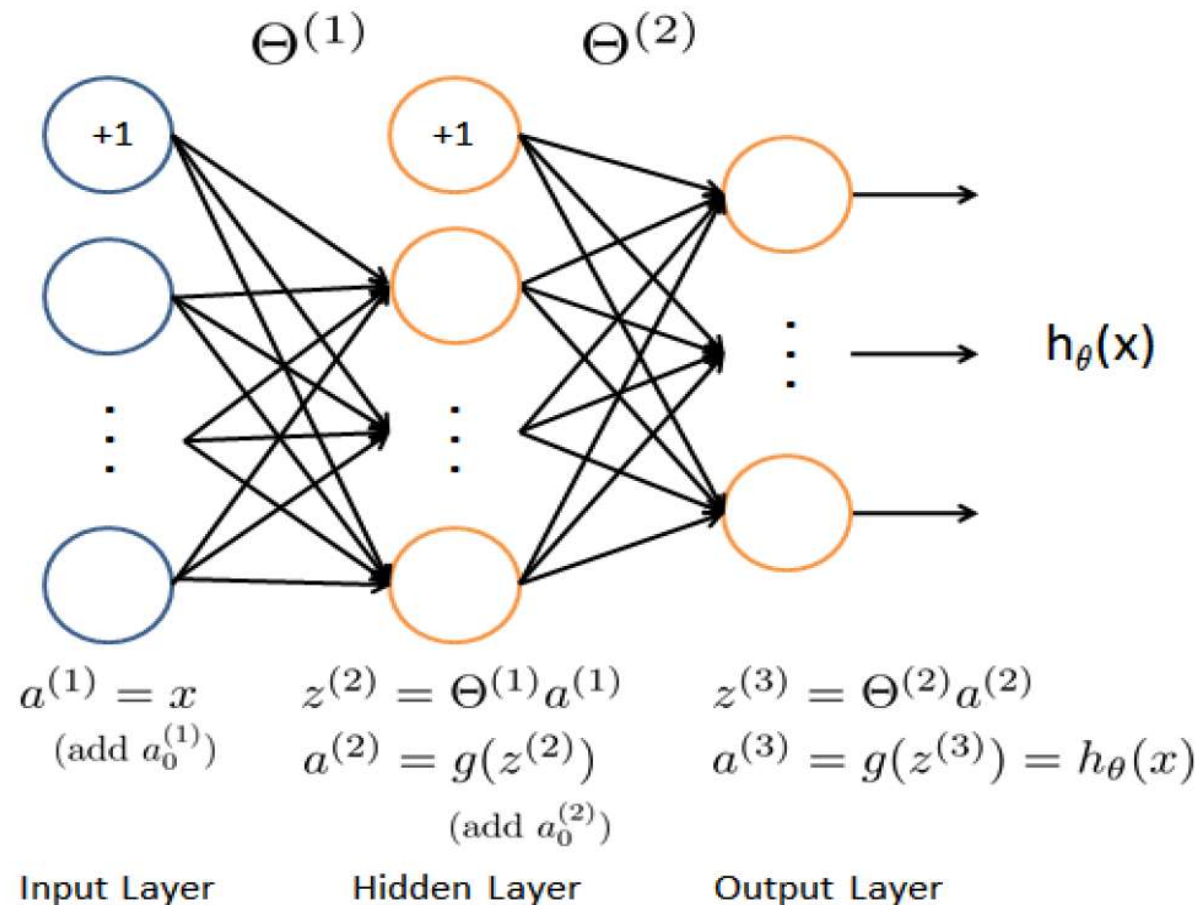
output layer - 10 output units (corresponding to 10 digit classes 0,1,2....9).

Matrix parameters: Θ_1 has size 25x401; Θ_2 has size 10x26.



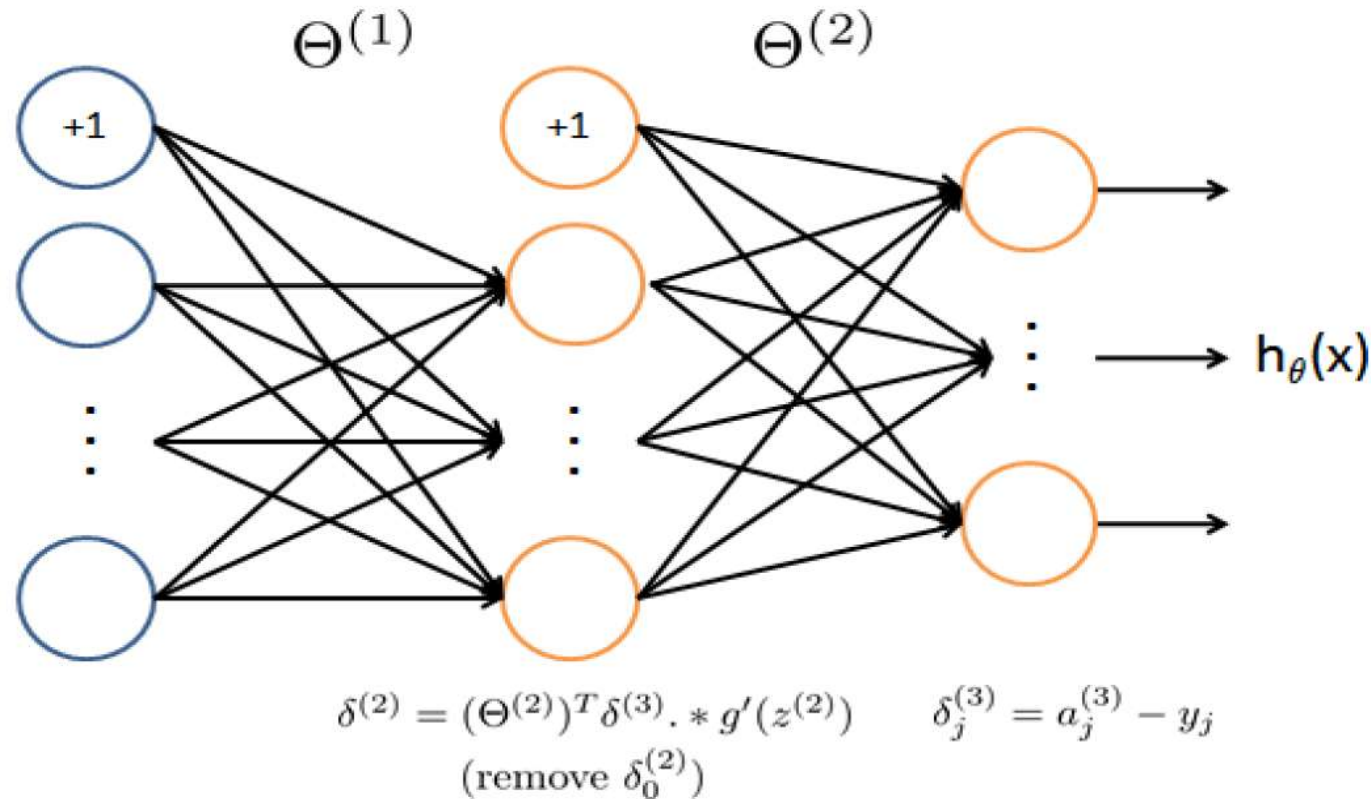
NN model learning – forward pass

- Randomly initialize the NN parameters (matrices Θ_1 and Θ_2).
- Provide features as inputs to the NN, make a forward pass to compute all activations through the NN and the NN outputs.
- Repeat for all examples (batch training)



NN model learning -Error Backpropagation

- Compute the output error (the difference between the NN output value and the true target value).
- For all hidden layer nodes compute an “error term” that measures how much that node was “responsible” for the NN output error.
- Compute the gradient as sum of the accumulated errors for all examples.
- Update the weights.



Error Backpropagation algorithm

- 0) Randomly initialize the parameters (matrices Θ_1 and Θ_2)
- 1) For $ii = 1:\text{number of examples } (m)$
- 2) Provide training example ii at the NN input.
- 3) Perform a feedforward pass to compute $z2$, $a2$ (for the hidden layer) and $z3$, $a3$ (for the output layer)

- 4) For each unit k in the output layer compute: $\delta_k^{(3)} = (a_k^{(3)} - y_k)$

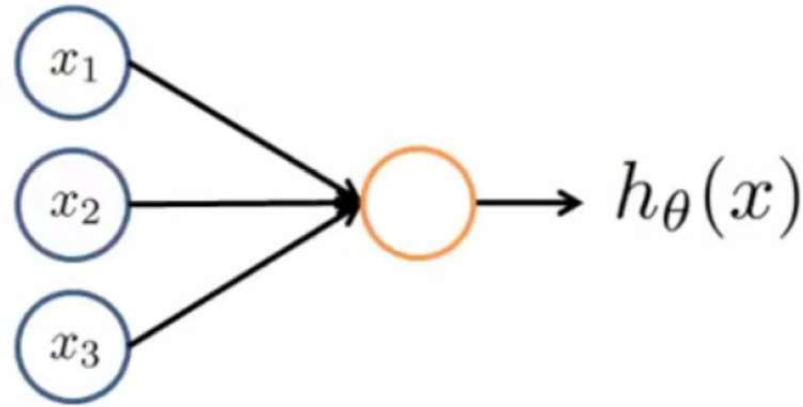
- 5) For the hidden layer, compute:
(error backpropagation) $\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} \cdot * g'(z^{(2)})$

- 6) Accumulate the gradient from this example: $\Delta^{(l)} = \Delta^{(l)} + \delta^{(l+1)} (a^{(l)})^T$

- 7) NN gradient (no regularization)
$$\frac{\partial J(\Theta)}{\partial \Theta_{ij}^{(l)}} = \frac{1}{m} \Delta_{ij}^{(l)}$$

- 8) Update NN parameters:
$$\Theta_{ij}^{(l)} = \Theta_{ij}^{(l)} - \alpha \frac{\partial J(\Theta)}{\partial \Theta_{ij}^{(l)}}$$

Sigmoid gradient



$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \quad \theta^T x = \theta_0 + \sum_{j=1}^n \theta_j x_j$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = \frac{d}{dz} g(z) = g(z)(1 - g(z))$$

Regularized Cost Function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left[-y_k^{(i)} \log((h_{\theta}(x^{(i)}))_k) - (1 - y_k^{(i)}) \log(1 - (h_{\theta}(x^{(i)}))_k) \right] +$$

Regularization term

$$\frac{\lambda}{2m} \left[\sum_{j=1}^{25} \sum_{k=1}^{400} (\Theta_{j,k}^{(1)})^2 + \sum_{j=1}^{10} \sum_{k=1}^{25} (\Theta_{j,k}^{(2)})^2 \right]$$

After computing the gradient by backpropagation, add the regularization term

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)} = \frac{1}{m} \Delta_{ij}^{(l)} \quad \text{for } j = 0$$

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)} = \frac{1}{m} \Delta_{ij}^{(l)} + \frac{\lambda}{m} \Theta_{ij}^{(l)} \quad \text{for } j \geq 1$$

Adaptive learning rate

$$\theta_j = \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

α - **Learning rate**

- **Fixed or**
- **Adaptive:**

$$\alpha^{(r+1)} = \begin{cases} b\alpha^{(r)} & \text{if } J^{(r+1)} \leq J^{(r)}, \quad b \geq 1 \text{ (ex. } b = 1.2) \\ b\alpha^{(r)} & \text{if } J^{(r+1)} > J^{(r)}, \quad b < 1 \text{ (ex. } b = 0.2) \end{cases} \quad \alpha^{(0)} = 0.01$$

Gradient Descent with momentum (extra term - momentum)

$$\theta_j^{(r)} = \theta_j^{(r-1)} - \alpha \frac{\partial J}{\partial \theta_j} + \beta (\theta_j^{(r-1)} - \theta_j^{(r-2)})$$

β - coefficient of momentum

- **Increase convergence rate far from minima**
- **Slow down near minima**

Gradient Descent with momentum is analogous to a ball moving on a surface with multiple valleys, accelerating on steep slides and decelerating when it reaches a valley.

The intuition behind is to add inertia to the gradient descent so that it smooth's the overall trajectory, in order to find better convergence points.

NN Parameters (weights) Initialization

- **Setting the weights to zero** (Simplest approach)

However, by initializing every weight to zero, every neuron will have the same activations, all the calculated gradients will be the same, and consequently, each parameter will suffer the same update. Therefore, it is crucial that the initialization of the weights breaks the symmetry between different units.

- **Drawn from random Gaussian distribution with mean 0 & deviation 1** may lead to vanishing gradients

Empirical initializations:

- **Xavier/ Glorot's initialization:** drawn from uniform distribution near zero.

$$\sim U\left(-\frac{\sqrt{6}}{\sqrt{m}}, \frac{\sqrt{6}}{\sqrt{m}}\right)$$

- **LeCun initialization:**

$$\sim U\left(-\frac{\sqrt{3}}{\sqrt{m}}, \frac{\sqrt{3}}{\sqrt{m}}\right)$$